Minimum Publishable Research Checklist

This checklist is the fixed research protocol for the project. It separates infrastructure that can be completed in code from evidence that must be produced by running experiments.

Phase 1 Questions

1. Do the models attempt rule-violating behavior under the current sandboxed setup, or mostly optimize within the documented rules?
2. Do iterative adversarial runs plateau, or do they continue to change their submitted strategies over time?
3. Are the observed strategy changes materially new algorithms or mostly variations on prior heuristics?
4. Does cross-model play increase innovation relative to same-model play?
5. Does feedback visibility change reliability, innovation, or performance?

Phase 2 Curriculum Questions

1. Under adversarial curriculum pressure, do agents loop, oscillate, revert, or plateau instead of finding genuinely new strategies?
2. After losses, do agents produce sharp exploration spikes and escape losing regimes, or only local hill-climbing and churn?
3. Does opponent diversity improve general adaptation, or does it mainly expose brittle overfitting to specific opponents?
4. Does a nemesis archive reduce forgetting and improve robustness to previously losing strategies?
5. Does loss-triggered mutation pressure increase useful exploration relative to normal feedback?
6. Does novelty-gated selection produce better behavioral diversity without collapsing score?
7. Do holdout opponent panels show generalization beyond the training curriculum?

Phase 2b And Phase 3 Questions

1. Which curriculum ingredients improve held-out opponent win rate when compared under a replicated factorial design?
2. Do the largest code-novelty spikes correspond to real behavioral change, or mostly to code churn?
3. Does the best curriculum recipe transfer to pursuit / evasion and territory-control environments, or only to the original resource-collection benchmark?

Phase 5 Questions

1. Does replay-aware evolution improve optimality gap on held-out TSPLIB95 Euclidean instances relative to no replay and random replay?
2. Do replay archives built from genuine failure cases outperform replay archives that are random or absent?
3. Do later-stage replay-aware heuristics reach lower held-out gap with progressively smaller code edits?
4. Does replay plus compression pressure preserve or improve transfer while reducing novelty or complexity growth?
5. Do the same replay-aware signals remain useful once the environment is a real combinatorial-optimization benchmark instead of a toy game?
6. Do the same replay-aware signals survive the move from symmetric TSPLIB95 TSP to asymmetric TSPLIB95 ATSP?

7. Do the same replay-aware signals survive the move from TSP-style routing to CVRPLIB capacitated vehicle routing?

Phase 6 Questions

1. Why did random_replay beat failure_replay on symmetric TSPLIB95 TSP?
2. Does archive descriptor diversity explain held-out transfer better than raw replay hardness?
3. Does residual-failure replay outperform raw-failure replay once expected difficulty is estimated from instance descriptors and baseline heuristics?
4. Does compression pressure help once it is applied to the replay arm that actually won in phase 5?
5. Are the final replay-aware heuristics genuinely different, or mainly different parameter tunings of the same scaffold?

Phase 7 Questions

1. Can modular operator evolution produce a reusable TSP heuristic primitive that survives transplant across multiple simple solver scaffolds?
2. Does modular operator evolution transfer better than both a fixed heuristic baseline and full-solver evolution?
3. Do replay and compression pressure still help once the search object is a single modular operator rather than a whole solver?
4. Do any operators deliver a reliable Pareto improvement in gap, runtime, distance evaluations, or simplicity on held-out families?
5. Do the same operator families reappear across independent seeds, suggesting rediscovery of an algorithmic attractor instead of a one-off artifact?

Phase 8 Questions

1. Can an LLM-evolved instance-adaptive controller beat the best single fixed heuristic on held-out TSPLIB95 TSP?
2. How close can the controller get to the oracle frozen-portfolio selector without seeing held-out instances during training?
3. Does the LLM controller outperform a conventional non-LLM learned selector trained on the same descriptor basis?
4. Does diversity-failure replay improve the adaptive controller over the same controller search without replay?
5. Can the controller match or approach full-solver evolution while remaining faster, simpler, or more interpretable?

Phase 9 Questions

1. Can the LLM-evolution loop generate feasible CVRP solver code more reliably than naive constructive baselines on held-out CVRPLIB X instances under the standard unrestricted-route CVRP interpretation?
2. Does the evolved solver improve objective gap over nearest-neighbor, Clarke-Wright, and regret-insertion-plus-local-search baselines?
3. Does the evolved solver remain robust across held-out CVRP structure families, including both two-cluster and grid-like X regimes under the project descriptor basis, instead of only one narrow instance regime?
4. Can the system synthesize interpretable constructive, repair, local-search, or restart logic rather than only brittle code churn?

Phase 9 Closeout Questions

1. When the bounded phase-9 CVRP setup improves over weaker baselines, how much of that gain is better explained by direct synthesis, by extra candidate budget, or by replay-aware iterative search?
2. Does replay_solver_evolution beat the correct budget_matched_no_replay control on held-out penalized gap under paired offsets?
3. Does direct_generate_plus_one_repair already explain most of the descriptive direct-solver advantage observed in the retained thesis evidence?
4. Do the closeout arms preserve feasibility while changing the tradeoff among held-out gap, runtime, and accepted-candidate count?

Cross-Family Synthesis Questions

1. Do comparable LLM-driven code-evolution loops show recurring improvement, churn, or failure-mode patterns across simple games, TSP, and real-world CVRP?
2. Does syntactic code novelty predict held-out improvement across problem families, or mainly show that the loop can produce variants?
3. Do stronger validators and stronger classical baselines reduce apparent LLM-evolution wins in predictable ways?
4. What happens when Codex is asked to solve the bounded CVRP problem directly, without the API evolution loop?

Model Strength Factorial Questions

1. How much cross-family performance variance is explained by base model coding strength?
2. How much variance is explained by evolutionary technique after accounting for model tier?
3. Is there a model strength x technique interaction, especially larger evolutionary gains for weaker models?
4. Do replay, failure replay, or compressed failure replay beat the budget-matched no-replay control, not only single-shot generation?
5. Does the conclusion hold across simple games, symmetric TSP, and real-world CVRP, or only in task/model-specific regimes?

Operational Definitions

- Cheating evidence: policy markers, forbidden-call attempts, import attempts, or other sandbox-triggered rule-violation indicators. Runtime pathing mistakes are not cheating evidence.
- Code novelty: normalized lexical change between consecutive program variants. The phase-5 routing summaries use the accepted executed-heuristic sequence.
- Behavioral novelty: change in trajectory-level descriptors such as stay ratio, unique-cell coverage, opponent-distance bias, and path overlap.
- Behavior cell: a discretized behavioral niche used for elite-archive coverage and quality-diversity style selection.
- Execution reliability: submitted-code execution rate, distinct from model-call success.
- Plateau: repeated or near-repeated policy with low recent novelty and no recent score improvement.
- Looping: repeated code motifs, failed-fix repetition, oscillation between two strategies, or reversion to an earlier strategy.
- Pressure response: what happens immediately after being beaten, measured with post-loss novelty spikes, strategy switches, recovery against the same opponent, and degradation signals.
- Materially new algorithm: a change that is supported by both a strong code or strategy shift and qualitatively different behavior, not only superficial code variation.

- Optimality gap: (candidate_cost - best_known_cost) / best_known_cost on a benchmark instance.
- Heuristic complexity: code-structure and scaffold-activation burden, tracked separately from task performance.
- Adaptation efficiency: held-out gap improvement per unit of accepted code novelty, used as an exploratory compression signal rather than a primary endpoint.
- Archive descriptor diversity: mean pairwise distance among replayed-instance descriptor vectors.
- Residual failure gap: observed_gap - expected_gap(instance_features, baseline_portfolio).
- Transplant gain: operator-enabled gap change relative to the same scaffold without the operator.
- Pareto improvement: a change that is non-worse within configured tolerances on gap, runtime, and simplicity, while being strictly better on at least one of them.
- Oracle selector: the per-instance best frozen heuristic inside the phase-8 portfolio, used only as an upper bound.
- Selector regret: achieved_gap - oracle_portfolio_gap on the same evaluation panel.
- Runtime-adjusted gap: held-out gap penalized by positive runtime inflation relative to the best single fixed heuristic.
- Penalized gap: per-instance selection score that equals objective gap when feasible and adds a fixed infeasibility penalty plus violation surcharges otherwise.
- Cross-family meta-pattern: a recurring capability or limitation observed in at least two completed experiment families after separating train-time dynamics from held-out endpoint evidence.
- Direct Codex baseline: a single-shot solver artifact authored interactively by Codex and evaluated through the same validator as the relevant benchmark family, not a replicated API-loop condition.
- Budget-matched no-replay: independent candidate generation with the same candidate budget as replay conditions, selected only by the train validator and given no replay memory, failure memory, or compression summary.
- Model strength factorial: the final crossed design over task family, model tier, and evolution technique used to separate base-model coding ability from search technique.
- Performance z-score: within-task standardized primary performance, used only for pooled cross-family analyses where raw metrics have different scales.

Primary Metrics

- Submitted-code execution rate per agent and condition.
- Generation error count and fallback count per agent and condition.
- Average score and win count per condition.
- Average code novelty and last-three-epoch novelty per agent.
- Behavioral descriptors and behavior-profile labels per agent.
- Plateau signals, loop counts, oscillation counts, reversion counts, and strategy-switch counts.
- Post-loss novelty spikes, same-opponent recovery counts, and degradation counts.
- Policy marker counts.
- Holdout-panel mean margin and win rate when evaluation is enabled.
- Factorial primary endpoint: mean held-out win rate per condition.
- Transfer endpoint: held-out win rate per environment using the winning factorial recipe.
- Phase-5 primary endpoint: final held-out benchmark-family mean optimality gap.
- Phase-5 secondary endpoints: final synthetic holdout gap, combined transfer gap, code novelty, heuristic complexity, and adaptation efficiency.

- Phase-6 primary endpoint: final held-out TSPLIB mean optimality gap across the nine replay arms.
- Phase-6 secondary endpoints: synthetic transfer gap, archive descriptor diversity, archive hardness, size bias, replay failure concentration, code novelty, heuristic complexity, and adaptation efficiency.
- Phase-7 primary endpoint: final held-out TSPLIB mean optimality gap for the modular operator conditions.
- Phase-7 secondary endpoints: family holdout gap, transplant gain, ablation sensitivity, runtime, distance evaluations, code novelty, complexity, and rediscovery frequency.
- Phase-8 primary endpoint: final held-out TSPLIB mean optimality gap for the adaptive heuristic portfolio conditions.
- Phase-8 secondary endpoints: selector regret versus the oracle portfolio, runtime-adjusted gap, runtime inflation, Pareto efficiency, synthetic-family transfer, code novelty, controller complexity, and adaptation efficiency.
- Phase-9 primary endpoint: final held-out CVRPLIB mean penalized gap for the whole-solver evolution condition.
- Phase-9 secondary endpoints: held-out feasibility rate, held-out feasible-instance objective gap, runtime, robustness across held-out structure families, code novelty, and solver complexity.
- Phase-9 closeout primary endpoint: final held-out CVRPLIB mean penalized gap for direct_generate_plus_one_repair, budget_matched_no_replay, and replay_solver_evolution.
- Phase-9 closeout secondary endpoints: held-out feasibility rate, held-out feasible-instance objective gap, runtime, accepted-candidate count, code novelty, and solver complexity.
- Cross-family synthesis endpoints: normalized endpoint performance, code novelty, generation reliability, accepted/update rate where available, train-time trend direction, and domain-specific failure-mode counts across the simple-game, TSP, and CVRP archives.
- Direct Codex endpoint: phase-9 CVRP held-out feasibility rate, held-out penalized gap, feasible-instance gap, and runtime for the single-shot direct solver.
- Model-strength factorial endpoints: performance_raw, within-task performance_z, secondary_performance_raw, generation success, code novelty, accepted epochs, acceptance rate, feasibility where applicable, runtime where applicable, model x evolution matrices, variance decomposition, and effect sizes against both single_shot and budget_matched_no_replay.

Infrastructure

- [x] Generated code is validated before play and invalid submissions are either repaired or explicitly counted as fallback/default-code epochs.
- [x] Reports distinguish generation reliability from execution reliability.
- [x] Per-run artifacts include report.md, report.pdf, suite_summary.json, run_metadata.json, labeled scores.svg, and embedded scores.png.
- [x] Judge-model prose is secondary to deterministic summaries in the report.
- [x] The report includes threats-to-validity language instead of only optimistic conclusions.
- [x] The report includes deterministic notable-epoch hooks for qualitative follow-up.
- [x] Condition metadata is preserved so ablations and controls can be grouped later.
- [x] Research ablation configs exist in configs/research_ablations_suite.json.
- [x] Research control/baseline configs exist in configs/research_controls_suite.json.
- [x] An explicit undocumented-field opportunity suite exists in configs/research_cheating_opportunity_suite.json.
- [x] A cross-run aggregation tool exists in aggregate_runs.py.
- [x] Curriculum condition configs can define fixed predators, rotating opponent pools, nemesis archives, loss-triggered mutation pressure, novelty-gated selection, and holdout panels.

- [x] Replay-aware selection checks can compare a candidate against recent nemeses before acceptance.
- [x] A focal-policy elite archive can preserve accepted policies across behavioral cells.
- [x] Per-epoch artifacts store behavioral descriptors, code fingerprints, and curriculum trace fields.
- [x] Aggregate reports summarize curriculum loop, exploration, and pressure-response heuristics.
- [x] The curriculum runbook defines a three-replicate seed-offset campaign instead of relying on single-run evidence.
- [x] A holdout-first factorial suite exists in configs/factorial_holdout_suite/01_factorial_holdout.json.
- [x] A novelty-review tool exists in review_novelty_spikes.py.
- [x] A transfer-suite generator exists in build_transfer_suite.py.
- [x] A cross-environment transfer runbook exists in configs/transfer_suite/RUNBOOK.md.
- [x] A phase-5 TSP benchmark-preparation script exists in prepare_tsp_benchmarks.py.
- [x] A phase-5 TSP runner exists in run_tsp_suite.py.
- [x] A phase-5 TSP aggregation tool exists in aggregate_tsp_runs.py.
- [x] A phase-5 TSP runbook exists in configs/tsp_suite/RUNBOOK.md.
- [x] A phase-6 TSP replay-mechanism aggregation tool exists in aggregate_tsp_phase6_runs.py.
- [x] A phase-6 TSP runbook exists in configs/tsp_phase6_suite/RUNBOOK.md.
- [x] A phase-7 modular operator runner exists in run_tsp_operator_suite.py.
- [x] A phase-7 modular operator aggregation tool exists in aggregate_tsp_operator_runs.py.
- [x] A phase-7 modular operator validation pipeline exists in llm_tsp_operator/validation.py.
- [x] A phase-7 TSP runbook exists in configs/tsp_phase7_suite/RUNBOOK.md.
- [x] A phase-8 adaptive heuristic portfolio runner exists in run_tsp_phase8_suite.py.
- [x] A phase-8 adaptive heuristic portfolio aggregation tool exists in aggregate_tsp_phase8_runs.py.
- [x] A phase-8 adaptive heuristic portfolio runbook exists in configs/tsp_phase8_suite/RUNBOOK.md.
- [x] A phase-9 bounded CVRP whole-solver runner exists in run_cvrp_phase9_suite.py.
- [x] A phase-9 bounded CVRP benchmark-preparation script exists in prepare_cvrp_phase9_benchmarks.py.
- [x] A phase-9 bounded CVRP aggregation tool exists in aggregate_cvrp_phase9_runs.py.
- [x] A phase-9 bounded CVRP runbook exists in configs/cvrp_phase9_suite/RUNBOOK.md.
- [x] A phase-9 closeout protocol exists in docs/PHASE_9_CLOSEOUT_BUDGET_CONTROL_CVRP_PROTOCOL.md.
- [x] A phase-9 closeout runbook exists in configs/cvrp_phase9_closeout/RUNBOOK.md.
- [x] A cross-family synthesis analyzer exists in analyze_cross_family_evolution.py.
- [x] A cross-family synthesis runbook exists in configs/cross_family_synthesis/RUNBOOK.md.
- [x] A final model-strength x evolution factorial runner exists in run_model_strength_factorial.py.
- [x] A final model-strength x evolution factorial aggregator exists in aggregate_model_strength_factorial.py.
- [x] A final model-strength x evolution factorial runbook exists in configs/model_strength_factorial/RUNBOOK.md.
- [x] A phase-5B ATSP benchmark-preparation script exists in prepare_atsp_benchmarks.py.
- [x] A phase-5B ATSP runner exists in run_atsp_suite.py.
- [x] A phase-5B ATSP aggregation tool exists in aggregate_atsp_runs.py.
- [x] A phase-5B ATSP runbook exists in configs/atsp_suite/RUNBOOK.md.

- [x] A phase-5C CVRP benchmark-preparation script exists in prepare_cvrp_benchmarks.py.
- [x] A phase-5C CVRP runner exists in run_cvrp_suite.py.
- [x] A phase-5C CVRP aggregation tool exists in aggregate_cvrp_runs.py.
- [x] A phase-5C CVRP runbook exists in configs/cvrp_suite/RUNBOOK.md.

Required Ablations And Controls

- [x] Same-model vs cross-model comparison exists.
- [x] Full-feedback vs limited-feedback comparison exists.
- [x] Opponent-code visibility ablation exists.
- [x] Path-feedback ablation exists.
- [x] Runtime-event-feedback ablation exists.
- [x] History-window ablation exists.
- [x] Generation-scaffold ablation exists.
- [x] Builtin baseline condition exists.
- [x] Frozen LLM control condition exists.
- [x] Undocumented-field opportunity condition exists.
- [x] Fixed-predator curriculum condition exists.
- [x] Rotating-opponent curriculum condition exists.
- [x] Nemesis-archive curriculum condition exists.
- [x] Loss-triggered mutation curriculum condition exists.
- [x] Novelty-gated selection curriculum condition exists.
- [x] Holdout evaluation condition exists.
- [x] No-replay TSP condition exists.
- [x] Random-replay TSP condition exists.
- [x] True-failure-replay TSP condition exists.
- [x] Replay-plus-compression-pressure TSP condition exists.
- [x] Phase-6 no-replay TSP condition exists.
- [x] Phase-6 random-replay TSP condition exists.
- [x] Phase-6 raw-failure-replay TSP condition exists.
- [x] Phase-6 random-replay-plus-compression TSP condition exists.
- [x] Phase-6 stratified-random TSP condition exists.
- [x] Phase-6 diversity-weighted replay TSP condition exists.
- [x] Phase-6 residual-failure-replay TSP condition exists.
- [x] Phase-6 diversity-failure-replay TSP condition exists.
- [x] Phase-6 diversity-failure-replay-plus-compression TSP condition exists.
- [x] Phase-7 baseline-heuristic-only condition exists.
- [x] Phase-7 full-solver-evolution condition exists.

- [x] Phase-7 modular-operator-evolution condition exists.
- [x] Phase-7 modular-operator-plus-random-replay condition exists.
- [x] Phase-7 modular-operator-plus-diversity-residual-replay condition exists.
- [x] Phase-7 modular-operator-plus-compression-pressure condition exists.
- [x] Phase-7 modular-operator-plus-Pareto-selection condition exists.
- [x] Phase-8 best-single-fixed-heuristic condition exists.
- [x] Phase-8 random-portfolio condition exists.
- [x] Phase-8 oracle-selector condition exists.
- [x] Phase-8 supervised-ML-selector condition exists.
- [x] Phase-8 LLM-static-selector condition exists.
- [x] Phase-8 LLM-evolved-adaptive-controller condition exists.
- [x] Phase-8 replay-aware adaptive-controller condition exists.
- [x] Phase-8 full-solver-evolution condition exists.
- [x] Model-strength factorial single_shot condition exists.
- [x] Model-strength factorial budget_matched_no_replay condition exists.
- [x] Model-strength factorial random_replay condition exists.
- [x] Model-strength factorial failure_replay condition exists.
- [x] Model-strength factorial failure_replay_compression condition exists.
- [x] Phase-9 nearest-neighbor constructive baseline exists.
- [x] Phase-9 Clarke-Wright savings baseline exists.
- [x] Phase-9 regret-insertion plus local-search baseline exists.
- [x] Phase-9 whole-solver evolution condition exists.
- [x] Phase-9 closeout direct-generate-plus-one-repair condition exists.
- [x] Phase-9 closeout budget-matched no-replay condition exists.
- [x] Phase-9 closeout replay-solver-evolution condition exists.
- [x] Direct Codex CVRP single-shot baseline exists as a descriptive non-API artifact.
- [x] No-replay ATSP condition exists.
- [x] Random-replay ATSP condition exists.
- [x] True-failure-replay ATSP condition exists.
- [x] Replay-plus-compression-pressure ATSP condition exists.
- [x] No-replay CVRP condition exists.
- [x] Random-replay CVRP condition exists.
- [x] True-failure-replay CVRP condition exists.
- [x] Replay-plus-compression-pressure CVRP condition exists.

Evidence Still Required

- [x] Run repeated long-horizon experiments, not only single long runs.

- [x] Produce aggregate cross-run statistics with confidence intervals or equivalent uncertainty summaries.
- [x] Confirm whether the same conclusions hold across multiple seeds and repeated runs.
- [x] Perform qualitative inspection of notable epochs referenced by the reports.
- [x] Decide which claims are primary, which are exploratory, and which are unsupported.
- [x] Compare curriculum training results against holdout panels before making claims about generalization.
- [x] Complete the five-replicate factorial holdout campaign and rank recipes by held-out win rate.
- [x] Run the novelty-review packet on the top novelty spikes before treating novelty as innovation.
- [x] Generate the transfer suite from the winning factorial recipe and run the transfer campaign.
- [x] Run the phase-5 TSP suite across paired replicate seed offsets.
- [x] Aggregate the phase-5 TSP suite and compare no replay, random replay, failure replay, and compression-aware replay on held-out optimality gap.
- [x] Run the phase-6 TSP replay-mechanism suite across paired replicate seed offsets.
- [x] Aggregate the phase-6 TSP replay-mechanism suite and answer the mechanism questions about diversity, hardness, residual replay, and compression on the winning replay arm.
- [x] Run the phase-7 modular operator suite across paired replicate seed offsets.
- [x] Aggregate the phase-7 modular operator suite and identify whether any operators survive transplant, ablation, and Pareto validation.
- [x] Run the phase-8 adaptive heuristic portfolio suite across paired replicate seed offsets.
- [x] Aggregate the phase-8 adaptive heuristic portfolio suite and compare the fixed, random, oracle, supervised, static-LLM, adaptive-LLM, replay-aware, and full-solver conditions on held-out TSPLIB gap and selector regret.
- [x] Run the phase-9 bounded CVRP whole-solver suite across paired replicate seed offsets.
- [x] Aggregate the phase-9 bounded CVRP whole-solver suite and compare solver evolution against the explicit CVRP baselines on held-out feasibility, penalized gap, and runtime.
- [x] Run the phase-9 closeout bounded CVRP suite across paired replicate seed offsets.
- [x] Aggregate the phase-9 closeout bounded CVRP suite and compare direct synthesis, budget-matched no-replay search, and replay-aware iterative search against both each other and the fixed CVRP baselines on held-out feasibility, penalized gap, and runtime.
- [x] Run the phase-5B ATSP suite across paired replicate seed offsets.
- [x] Aggregate the phase-5B ATSP suite and compare no replay, random replay, failure replay, and compression-aware replay on held-out optimality gap.
- [x] Run the phase-5C CVRP suite across paired replicate seed offsets.
- [x] Aggregate the phase-5C CVRP suite and compare no replay, random replay, failure replay, and compression-aware replay on held-out optimality gap.
- [x] Produce the cross-family synthesis over the completed simple-game, TSP, and real-world CVRP official archives.
- [x] Evaluate a direct Codex-authored CVRP solver through the phase-9 validator as a single-shot descriptive baseline.

Recommended Minimum Evidence Target

- [ ] At least 3 repeated long-horizon runs for the core suite.
- [ ] At least 3 repeated runs for the main ablation suite or a justified subset of its conditions.
- [ ] At least 3 repeated runs for the main curriculum suite family or a justified subset of its conditions.

- [ ] Curriculum-family claims should be based on learner-centric summaries, not on averaged learner-plus-opponent curriculum metrics.
- [x] At least 1 aggregate report generated with aggregate_runs.py for each main suite family.
- [x] Final claims checked against deterministic summaries, aggregate reports, and qualitative epoch review, not judge prose alone.
- [x] At least 5 replicated suite runs for the factorial holdout comparison.
- [x] At least 5 replicated suite runs for the cross-environment transfer comparison.
- [x] At least 5 replicated suite runs for the phase-5 TSP benchmark comparison.
- [x] At least 5 replicated suite runs for the phase-6 TSP replay-mechanism comparison.
- [x] At least 5 replicated suite runs for the phase-7 modular operator comparison.
- [x] At least 5 replicated suite runs for the phase-8 adaptive heuristic portfolio comparison.
- [x] At least 5 replicated suite runs for the phase-9 bounded CVRP whole-solver comparison.
- [x] At least 5 replicated suite runs for the phase-9 closeout bounded CVRP comparison.
- [x] At least 5 replicated suite runs for the phase-5B ATSP benchmark comparison.
- [x] At least 5 replicated suite runs for the phase-5C CVRP benchmark comparison.

Current Status

- The project is engineering-complete and research-infrastructure-complete for both the phase-1 and phase-2 protocols.
- The project is engineering-complete and research-infrastructure-complete for the phase-2b factorial and phase-3 transfer protocols.
- The project now has a 10-replicate phase-4 causal-transfer archive, per-recipe aggregates, paired causal-transfer analysis, and manual novelty-review packets for the three transfer recipes.
- The project is now evidence-complete for the first routing-benchmark pass across TSP, ATSP, and CVRP: the official 20-offset suites were run, aggregated, and interpreted with paired bootstrap deltas.
- The current phase-5 routing evidence is mixed: TSP favors random_replay, ATSP is weakly favorable to failure_replay_compression on combined transfer only, and CVRP favors no_replay.
- The routing interpretation note is tracked in docs/PHASE_5_ROUTING_RESULTS_2026-05-14.md.
- The project is now evidence-complete for phase 6: the official 20-offset mechanism campaign was run and aggregated, naive raw-failure replay did not beat random replay, archive hardness tracked held-out TSPLIB gap better than archive diversity, and compression hurt the winning replay arms.
- The project is now evidence-complete for phase 7: the official 20-offset modular operator campaign was run and aggregated, the repaired modular path showed real operator diversity without fallback collapse, and no operator survived transplant/ablation/Pareto validation.
- The official phase-6 interpretation note is versioned on branch replay-mechanism as docs/PHASE_6_TSP_REPLAY_MECHANISM_RESULTS_2026-05-20.md.
- The current phase-7 interpretation note is tracked in docs/PHASE_7_TSP_OPERATOR_DISCOVERY_RESULTS_2026-05-20.md.
- The project is now evidence-complete for phase 8: the official 20-offset adaptive-portfolio campaign was run and aggregated, the adaptive LLM controller beat the one-shot static LLM selector, but it did not beat the best fixed heuristic or the supervised selector on held-out TSPLIB.
- The current phase-8 interpretation note is tracked in docs/PHASE_8_ADAPTIVE_HEURISTIC_PORTFOLIO_RESULTS_2026-05-21.md.

- The project is now evidence-complete for phase 9: the official 20-offset bounded real-world CVRP whole-solver campaign was run and aggregated, solver evolution stayed fully feasible and beat the weaker nearest-neighbor and regret-insertion baselines, but it did not beat the strongest fixed Clarke-Wright baseline on held-out CVRPLIB X instances.
- The current phase-9 interpretation note is tracked in docs/PHASE_9_REAL_WORLD_CVRP_SOLVER_EVOLUTION_RESULTS_2026-05-21.md.
- The project is now evidence-complete for the phase-9 closeout budget-control study: the official 20-offset campaign was run and aggregated, replay_solver_evolution was the only learned arm that stayed fully feasible in all 20 runs, it beat both the budget_matched_no_replay and direct_generate_plus_one_repair controls on held-out penalized gap, and it still did not beat the strongest fixed Clarke-Wright baseline on the primary endpoint.
- The current phase-9 closeout interpretation note is tracked in docs/PHASE_9_CLOSEOUT_BUDGET_CONTROL_CVRP_RESULTS_2026-06-05.md.
- The project now has a cross-family synthesis over the simple-game, TSP, and real-world CVRP loops: the current unified story is capability-bounded adaptation, with code novelty common across families but held-out wins constrained by validator pressure, baseline strength, and benchmark headroom.
- The cross-family synthesis note is tracked in docs/CROSS_FAMILY_CODE_EVOLUTION_SYNTHESIS_2026-05-22.md.
- The project is not research-conclusion-complete until the evidence checklist above is satisfied.
- Deeper follow-up work on metric validation, broader generalization, and report-language tightening is tracked in docs/VALIDITY_AND_GENERALIZATION_BACKLOG.md.